



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

编译系统原理实验报告

了解编译器 & LLVM IR 编程 & 汇编编程

刘宇泽 孙锴梵

年级：2024 级

专业：计算机科学与技术

指导教师：王刚

2026 年 10 月 4 日

目录

一、 实验介绍	1
(一) 实验目的	1
(二) 问题描述	1
(三) 分工	1
二、 实验内容	2
(一) 了解编译器工作过程	2
1. 完整的编译过程	2
2. 预处理器	3
3. 编译器	5
4. 汇编器	22
5. 链接器	24
(二) 熟悉 LLVM IR 中间语言	27
1. 实验程序	27
2. 编译与链接	28
3. 运行验证	28
(三) 熟悉汇编语言	29
三、 总结	34

一、 实验介绍

（一） 实验目的

- **了解完整的编译流程：**以 GCC、LLVM 等熟悉编译器为对象，探究一个程序从源代码到可执行程序完整处理过程。
- **理解预处理器的作用：**了解预处理器如何处理以 # 开头的指令，如 #include、#define、条件编译等，以及宏展开、头文件插入、注释删除等操作。
- **理解编译器的作用：**了解编译器如何完成词法分析、语法分析、语义分析、中间代码生成、代码优化和代码生成等工作。
- **理解汇编器的作用：**了解汇编器如何把汇编语言翻译成机器指令，并生成可重定位目标文件。
- **理解链接器的作用：**了解链接器如何把多个目标文件和库文件链接起来，形成可以真正运行的可执行程序。
- **熟悉 LLVM IR 中间语言：**通过编写和阅读 LLVM IR 程序，理解 LLVM IR 的基本结构、表示形式和主要特点，例如类型化、可扩展性和较强的表达能力。
- **熟悉 ARM/RISC-V 汇编语言：**通过编写汇编程序，掌握基本汇编指令、寄存器、标签、段定义、函数调用约定和栈帧等内容，并尽量了解其底层实现方式。

（二） 问题描述

任务一，以 GCC 为研究对象，深入地探究语言处理系统的完整工作过程，其中包括：完整的编译过程、预处理器的任务、编译器的任务、汇编器的任务、链接器的任务、如何执行程序。

任务二，通过编写 LLVM IR 程序，熟悉 LLVM IR 中间语言。

任务三，通过编写汇编程序，熟悉 RISC-V 汇编语言。

（三） 分工

选用 risc-v 作为实验框架

刘宇泽（2411334）负责完成**任务一**的完整的编译过程、预处理器的任务、编译器的任务以及**任务二**。

孙铭梵（2410868）负责完成**任务一**的汇编器的任务、链接器的任务以及**任务三**。

二、 实验内容

(一) 了解编译器工作过程

1. 完整的编译过程

一个完整的编译过程主要分为以下几个过程：

- **预处理**：处理以 # 开头的预处理指令，例如展开宏、插入头文件、处理条件编译，并删除注释、补充行号和文件名信息，生成预处理后的源程序。
- **编译**：把预处理后的源程序翻译成汇编代码。编译器内部依次进行词法分析、语法分析、语义分析、中间代码生成、代码优化和代码生成。
- **汇编**：把汇编代码翻译成机器指令，并生成包含符号表和重定位信息的可重定位目标文件。
- **链接**：解析外部符号、完成地址重定位，并把多个目标文件和库文件组合成可执行文件。
- **加载与运行**：程序执行时，加载器把可执行文件装入内存，完成运行前的准备工作。

流程如图1所示：

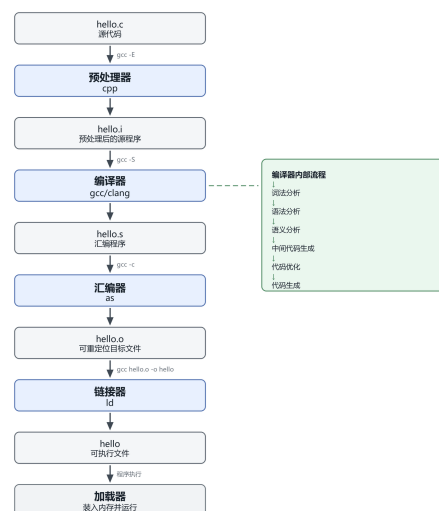


图 1: 编译过程

编写 main.c 文件来对编译过程进行测试，程序源码如下所示：

```

1 main()
2 {
3     int a,b,i,t,n;
4     a=0;
5     b=1;
6     i=1;
7     cin>>n;
8     cout<<a<<endl;
9     cout<<b<<endl;
  
```

```
10     while(i<n)
11     {
12         t=b;
13         b=a+b;
14         cout<<b<<endl;
15         a=t;
16         i=i+1;
17     }
18 }
```

使用 `clang -ccc-print-phases main.c` 指令，得到编译的整个流程，如图2所示。

```
liuyuze@WIN-HBB36NFFET0:~/lab/lab1$ clang -ccc-print-phases main.c
+- 0: input, "main.c", c
+- 1: preprocessor, {0}, cpp-output
+- 2: compiler, {1}, ir
+- 3: backend, {2}, assembler
+- 4: assembler, {3}, object
5: linker, {4}, image
```

图 2: main.c 编译

完全符合我们所学到的编译流程。

2. 预处理器

(1) 预处理阶段的功能

- 处理预处理指令

处理源代码中以 `#` 开头的指令，例如 `#include`、`#define`、`#if` 等。

- 展开头文件和宏定义

对于 `#include`，将对应头文件的内容插入源文件；对于 `#define`，将宏在代码中出现的位置替换为相应内容。

- 处理条件编译

根据 `#if` 等条件编译指令，决定哪些代码参与后续编译、哪些代码被忽略。

- 清理和标记源代码

删除源代码中的注释，并添加行号、文件名等标识，以便后续编译器定位和处理代码。

- 生成预处理后的源程序

预处理器的输出不是汇编代码或机器代码，而是经过头文件插入、宏展开和条件选择后的源程序。

- 作为编译流程的第一个阶段

预处理结果会交给编译器继续进行词法分析、语法分析、语义分析、中间代码生成等工作。

(2) 实验验证

为了验证预处理器的主要功能，本实验在原斐波那契数列程序的基础上进行了修改。首先，在源文件开头加入 `#include <stdio.h>`，以使用 C 语言中的 `scanf` 和 `printf` 完成输入输出。其次，使用 `#define` 定义宏常量 `FIB_A0`、`FIB_B0`、`FIB_IO` 以及函数宏 `FIB_NEXT(x)`，并将程序中的相关赋值和循环变量更新替换为这些宏，以便观察宏替换的结果。同时，在代码中加入单行注释和多行注释，用于验证预处理器对注释的处理；并通过 `#ifdef FIB_DEBUG` 加入调试输出语句，用于验证条件编译功能。

修改后的代码如下所示

```
1  #include <stdio.h>
2
3  // FIB_SINGLE_COMMENT
4
5  /* FIB_BLOCK_COMMENT */
6
7  #define FIB_A0 0
8  #define FIB_B0 1
9  #define FIB_IO 1
10 #define FIB_NEXT(x) ((x) + 1)
11
12 int main()
13 {
14     int a, b, i, t, n;
15
16     a = FIB_A0;
17     b = FIB_B0;
18     i = FIB_IO;
19
20     scanf("%d", &n);
21
22     printf("%d\n", a);
23     printf("%d\n", b);
24
25     #ifdef FIB_DEBUG
26         printf("debug\n");
27     #endif
28
29     while (i < n)
30     {
31         t = b;
32         b = a + b;
33
34         printf("%d\n", b);
35
36         a = t;
```

```

37         i = FIB_NEXT(i);
38     }
39
40     return 0;
41 }

```

完成修改后，执行

```
gcc main.c -E -o main.i
```

得到预处理后的文件 `main.i`。通过对比源文件和预处理结果，可以进一步分析头文件展开、宏替换、注释删除以及条件编译等预处理功能：

```

C main: X
120 void *C_main()
121 {
122     .....
123     # 30 "/usr/include/stdint.h" 2 3 4
124     # 1 "/usr/include/x86_64-linux-gnu/bits/types/__fpas_t.h" 1 3 4
125
126
127
128
129     # 1 "/usr/include/x86_64-linux-gnu/bits/types/__mstate_t.h" 1 3 4
130     # 13 "/usr/include/x86_64-linux-gnu/bits/types/__mstate_t.h" 1 3 4
131     typedef struct
132     {
133     {
134         int __count;
135         union
136         {
137             unsigned int __wch;
138             char __wchb[4];
139         } __value;
140     } __mstate_t;
141     # 6 "/usr/include/x86_64-linux-gnu/bits/types/__fpas_t.h" 2 3 4
142
143
144
145
146
147
148
149
150
151     typedef struct __fpas_t
152     {
153     {
154         __off_t __pos;
155         __mstate_t __state;
156     } __fpas_t;
157     # 48 "/usr/include/stdint.h" 2 3 4
158     # 17 1 "/usr/include/x86_64-linux-gnu/bits/types/__fpas64_t.h" 1 3 4
159     # 10 "/usr/include/x86_64-linux-gnu/bits/types/__fpas64_t.h" 1 3 4
160     typedef struct __fpas64_t
161     {
162     {
163         __off64_t __pos;
164         __mstate_t __state;
165     } __fpas64_t;
166     # 41 "/usr/include/stdint.h" 2 3 4
167     # 1 "/usr/include/x86_64-linux-gnu/bits/types/__FILE.h" 1 3 4
168
169
170
171
172
173
174
175
176
177
178
179     struct _IO_FILE;
180     typedef struct _IO_FILE __FILE;
181     # 42 "/usr/include/stdint.h" 2 3 4
182     # 1 "/usr/include/x86_64-linux-gnu/bits/types/FILE.h" 1 3 4
183
184
185
186
187
188
189
190
191
192
193
194
195     struct _IO_FILE;
196
197
198
199
200
201
202
203     typedef struct _IO_FILE FILE;
204     # 43 "/usr/include/stdint.h" 2 3 4
205     # 1 "/usr/include/x86_64-linux-gnu/bits/types/struct_FILE.h" 1 3 4
206     # 10 "/usr/include/x86_64-linux-gnu/bits/types/struct_FILE.h" 1 3 4
207     struct _IO_FILE;
208     struct _IO_marker;
209     struct _IO_codecvt;
210     struct _IO_wide_data;
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251     typedef void _IO_lock_t;
252

```

图 3: 预处理后的代码

3. 编译器

(1) 词法分析

词法分析的主要功能是：从左到右扫描预处理后的源程序，将其中的字符序列划分为具有明确类别的词法单元（Token），例如关键字、标识符、常量、运算符和分隔符，并记录它们在源程序中的位置，为后续的语法分析提供输入。

本节使用经过预处理的 `main.c`，执行如下命令：

```
clang -E -Xclang -dump-tokens main.c
```

该命令会在预处理的基础上输出词法分析结果。部分结果如图 4 所示。

```

liuyuze@WIN-HB836NFFET0:~/lab/lab1$ clang -E -Xclang -dump-tokens main.c
typedef 'typedef' [StartOfLine] Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stddef.h:46:1>
long 'long' [LeadingSpace] Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stddef.h:46:9> <Spelling=<built-in>:92:23>
unsigned 'unsigned' [LeadingSpace] Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stddef.h:46:9> <Spelling=<built-in>:92:28>
int 'int' [LeadingSpace] Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stddef.h:46:9> <Spelling=<built-in>:92:37>
identifier 'size_t' [LeadingSpace] Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stddef.h:46:23>
semi ';' Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stddef.h:46:29>
typedef 'typedef' [StartOfLine] Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stdarg.h:14:1>
identifier 'builtin_va_list' [LeadingSpace] Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stdarg.h:14:9>
identifier 'va_list' [LeadingSpace] Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stdarg.h:14:27>
semi ';' Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stdarg.h:14:34>
typedef 'typedef' [StartOfLine] Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stdarg.h:32:1>
identifier 'builtin_va_list' [LeadingSpace] Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stdarg.h:32:9>
identifier '__gnuc_va_list' [LeadingSpace] Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stdarg.h:32:27>
semi ';' Loc=</usr/lib/llvm-14/lib/clang/14.0.0/include/stdarg.h:32:41>
typedef 'typedef' [StartOfLine] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:31:1>
unsigned 'unsigned' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:31:9>
char 'char' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:31:18>
identifier '__u_char' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:31:23>
semi ';' Loc=</usr/include/x86_64-linux-gnu/bits/types.h:31:31>
typedef 'typedef' [StartOfLine] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:32:1>
unsigned 'unsigned' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:32:9>
short 'short' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:32:18>
int 'int' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:32:24>
identifier '__u_short' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:32:28>
semi ';' Loc=</usr/include/x86_64-linux-gnu/bits/types.h:32:37>
typedef 'typedef' [StartOfLine] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:33:1>
unsigned 'unsigned' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:33:9>
int 'int' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:33:18>
identifier '__u_int' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:33:22>
semi ';' Loc=</usr/include/x86_64-linux-gnu/bits/types.h:33:28>
typedef 'typedef' [StartOfLine] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:34:1>
unsigned 'unsigned' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:34:9>
long 'long' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:34:18>
int 'int' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:34:23>
identifier '__u_long' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:34:27>
semi ';' Loc=</usr/include/x86_64-linux-gnu/bits/types.h:34:35>
typedef 'typedef' [StartOfLine] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:37:1>
signed 'signed' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:37:9>
char 'char' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:37:16>
identifier '__int8_t' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:37:21>
semi ';' Loc=</usr/include/x86_64-linux-gnu/bits/types.h:37:28>
typedef 'typedef' [StartOfLine] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:38:1>
unsigned 'unsigned' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:38:9>
char 'char' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:38:18>
identifier '__uint8_t' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:38:23>
semi ';' Loc=</usr/include/x86_64-linux-gnu/bits/types.h:38:32>
typedef 'typedef' [StartOfLine] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:39:1>
signed 'signed' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:39:9>
short 'short' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:39:16>
int 'int' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:39:22>
identifier '__int16_t' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:39:26>
semi ';' Loc=</usr/include/x86_64-linux-gnu/bits/types.h:39:35>
typedef 'typedef' [StartOfLine] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:40:1>
unsigned 'unsigned' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:40:9>
short 'short' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:40:18>
int 'int' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:40:24>
identifier '__uint16_t' [LeadingSpace] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:40:28>
semi ';' Loc=</usr/include/x86_64-linux-gnu/bits/types.h:40:38>
typedef 'typedef' [StartOfLine] Loc=</usr/include/x86_64-linux-gnu/bits/types.h:41:1>

```

图 4: 词法分析结果

以下截取 main 函数附近的典型部分进行分析。

i. 主函数与变量声明

```

1  int 'int' [StartOfLine] Loc=<main.c:12:1>
2  identifier 'main' [LeadingSpace] Loc=<main.c:12:5>
3  l_paren '(' Loc=<main.c:12:9>
4  r_paren ')' Loc=<main.c:12:10>
5  l_brace '{' [StartOfLine] Loc=<main.c:13:1>
6  int 'int' [StartOfLine] [LeadingSpace] Loc=<main.c:14:5>
7  identifier 'a' [LeadingSpace] Loc=<main.c:14:9>
8  comma ',' Loc=<main.c:14:10>
9  identifier 'b' [LeadingSpace] Loc=<main.c:14:12>
10 comma ',' Loc=<main.c:14:13>
11 identifier 'i' [LeadingSpace] Loc=<main.c:14:15>
12 comma ',' Loc=<main.c:14:16>
13 identifier 't' [LeadingSpace] Loc=<main.c:14:18>
14 comma ',' Loc=<main.c:14:19>
15 identifier 'n' [LeadingSpace] Loc=<main.c:14:21>
16 semi ';' Loc=<main.c:14:22>

```

这里, int 被识别为关键字, main、a、b、i、t 和 n 被识别为标识符; 左右括号、左右花括号分别用于表示参数列表和函数体, 逗号表示变量之间的分隔, 分号表示一条声明语句的结束。

ii. 赋值、输入输出与运算符

1	identifier 'a'	[StartOfLine] [LeadingSpace] Loc=<main.c:16:5>
2	equal '='	[LeadingSpace] Loc=<main.c:16:7>
3	numeric_constant '0'	[LeadingSpace]
4		Loc=<main.c:16:9 <Spelling=main.c:7:16>>
5	semi ';'	Loc=<main.c:16:15>
6	identifier 'scanf'	[StartOfLine] [LeadingSpace] Loc=<main.c:20:5>
7	l_paren '('	Loc=<main.c:20:10>
8	string_literal '%"d"'	Loc=<main.c:20:11>
9	comma ','	Loc=<main.c:20:15>
10	amp '&'	[LeadingSpace] Loc=<main.c:20:17>
11	identifier 'n'	Loc=<main.c:20:18>
12	r_paren ')'	Loc=<main.c:20:19>
13	semi ';'	Loc=<main.c:20:20>
14	identifier 'printf'	[StartOfLine] [LeadingSpace] Loc=<main.c:22:5>
15	l_paren '('	Loc=<main.c:22:11>
16	string_literal '%"d\n"'	Loc=<main.c:22:12>
17	comma ','	Loc=<main.c:22:18>
18	identifier 'a'	[LeadingSpace] Loc=<main.c:22:20>
19	r_paren ')'	Loc=<main.c:22:21>
20	semi ';'	Loc=<main.c:22:22>

赋值号被识别为 equal, 加号被识别为 plus; 取地址符号被识别为 amp。scanf 和 printf 是标识符, 而 "%d" 与 "%d\n" 是完整的字符串常量。后者中的 \n 属于字符串字面量的一部分, 不会被拆分成独立的词法单元。

宏常量 FIB_A0 已在此前的预处理阶段被替换为数值常量 0。输出中的 Spelling=main.c:7:16 说明了该数值来源于 #define FIB_A0 0 所在的位置, 因此词法分析器接收到的内容已经是宏展开后的代码。

iii. 循环、宏函数展开与程序结束

1	while 'while'	[StartOfLine] [LeadingSpace] Loc=<main.c:29:5>
2	l_paren '('	[LeadingSpace] Loc=<main.c:29:11>
3	identifier 'i'	Loc=<main.c:29:12>
4	less '<'	[LeadingSpace] Loc=<main.c:29:14>
5	identifier 'n'	[LeadingSpace] Loc=<main.c:29:16>
6	r_paren ')'	Loc=<main.c:29:17>
7	l_brace '{'	[StartOfLine] [LeadingSpace] Loc=<main.c:30:5>
8	identifier 't'	[StartOfLine] [LeadingSpace] Loc=<main.c:31:9>
9	equal '='	[LeadingSpace] Loc=<main.c:31:11>
10	identifier 'b'	[LeadingSpace] Loc=<main.c:31:13>
11	semi ';'	Loc=<main.c:31:14>
12	identifier 'b'	[StartOfLine] [LeadingSpace] Loc=<main.c:32:9>
13	equal '='	[LeadingSpace] Loc=<main.c:32:11>
14	identifier 'a'	[LeadingSpace] Loc=<main.c:32:13>
15	plus '+'	[LeadingSpace] Loc=<main.c:32:15>
16	identifier 'b'	[LeadingSpace] Loc=<main.c:32:17>
17	semi ';'	Loc=<main.c:32:18>
18	identifier 'i'	[StartOfLine] [LeadingSpace] Loc=<main.c:37:9>
19	equal '='	[LeadingSpace] Loc=<main.c:37:11>
20	l_paren '('	[LeadingSpace]
21		Loc=<main.c:37:13 <Spelling=main.c:10:21>>
22	l_paren '('	Loc=<main.c:37:13 <Spelling=main.c:10:22>>
23	identifier 'i'	Loc=<main.c:37:13 <Spelling=main.c:37:22>>
24	r_paren ')'	Loc=<main.c:37:13 <Spelling=main.c:10:24>>

```

25 plus '+' [LeadingSpace]
26 Loc=<main.c:37:13 <Spelling=main.c:10:26>>
27 numeric_constant '1' [LeadingSpace]
28 Loc=<main.c:37:13 <Spelling=main.c:10:28>>
29 r_paren ')' Loc=<main.c:37:13 <Spelling=main.c:10:29>>
30 semi ';' Loc=<main.c:37:24>
31 return 'return' [StartOfLine] [LeadingSpace] Loc=<main.c:40:5>
32 numeric_constant '0' [LeadingSpace] Loc=<main.c:40:12>
33 semi ';' Loc=<main.c:40:13>
34 r_brace '}' [StartOfLine] Loc=<main.c:41:1>
35 eof '' Loc=<main.c:41:2>

```

`while` 和 `return` 被识别为关键字,小于号被识别为 `less`,加号被识别为 `plus`。`FIB_NEXT(i)` 在预处理阶段被展开为 `((i) + 1)`, 因此词法分析结果中出现的是括号、标识符 `i`、加号和数值常量 `1`, 而不是单个名为 `FIB_NEXT` 的词法单元。

输出中没有出现单行注释、多行注释以及 `#ifdef FIB_DEBUG` 控制的调试输出。这是因为本节执行的是预处理后的词法分析: 注释已经在预处理阶段被删除, 而未定义 `FIB_DEBUG` 时, 调试输出语句也不会进入预处理结果。最后的 `eof` 表示词法分析器已经扫描到输入文件的末尾。

综上, 源程序被成功分解为具有明确类别和位置信息的词法单元; 宏定义和条件编译已在词法分析之前处理, 函数宏 `FIB_NEXT(i)` 也被展开为普通的括号、运算符和常量序列。由此验证了词法分析将字符流转换为 Token 序列的功能。

(2) 语法分析

语法分析的主要功能是: 以前一步词法分析产生的 Token 序列作为输入, 根据 C 语言的语法规则识别程序的层次结构, 并构造抽象语法树 (Abstract Syntax Tree, AST)。使用如下命令生成 AST:

```
clang -E -Xclang -ast-dump main.c
```

终端部分结果如图 5 所示。

```

liuyuzhe@WIN-HB836NFFET0:~/lab/lab1$ clang -E -Xclang -ast-dump main.c
translationUnitDecl @0x732a08 <invalid sloc> <invalid sloc>
-FunctionDecl @0x732338 <invalid sloc> <invalid sloc> implicit __int128_t 'int128'
  -BuiltinType @0x732fd0 'int128'
-FunctionDecl @0x732a08 <invalid sloc> <invalid sloc> implicit __uint128_t 'unsigned __int128'
  -BuiltinType @0x732ff0 'unsigned __int128'
-FunctionDecl @0x7335a8 <invalid sloc> <invalid sloc> implicit __NSConstantString_tag 'struct __NSConstantString_tag'
  -RecordType @0x733388 'struct __NSConstantString_tag'
    -Record @0x7332f8 ' __NSConstantString_tag'
-FunctionDecl @0x733640 <invalid sloc> <invalid sloc> implicit __builtin_ms_va_list 'char **'
  -PointerType @0x733600 'char *'
    -BuiltinType @0x732ab0 'char'
-FunctionDecl @0x733938 <invalid sloc> <invalid sloc> implicit referenced __builtin_va_list 'struct __va_list_tag[1]'
  -ConstantArrayType @0x7338e0 'struct __va_list_tag[1]' 1
    -RecordType @0x733720 'struct __va_list_tag'
      -Record @0x733698 ' __va_list_tag'
-FunctionDecl @0x7339a8 </usr/lib/llvm-14/lib/clang/14.0.0/include/stddef.h:46:1, col:23> col:23 referenced size_t 'unsigned long'
  -BuiltinType @0x732bd0 'unsigned long'
-FunctionDecl @0x73bd70 </usr/lib/llvm-14/lib/clang/14.0.0/include/stdarg.h:14:1, col:27> col:27 va_list ' __builtin_va_list': 'struct __va_list_tag[1]'
  -FunctionDecl @0x73bd40 ' __builtin_va_list' sugar
    -TypeDecl @0x733938 ' __builtin_va_list'
      -ConstantArrayType @0x7338e0 'struct __va_list_tag[1]' 1
        -RecordType @0x733720 'struct __va_list_tag'
          -Record @0x733698 ' __va_list_tag'
-FunctionDecl @0x73bd40 </usr/lib/llvm-14/lib/clang/14.0.0/include/stdarg.h:14:1, col:27> col:27 referenced __gnuc_va_list ' __builtin_va_list': 'struct __va_list_tag[1]'
  -TypeDecl @0x73bd40 ' __builtin_va_list' sugar
    -ConstantArrayType @0x7338e0 'struct __va_list_tag[1]' 1
      -RecordType @0x733720 'struct __va_list_tag'
        -Record @0x733698 ' __va_list_tag'
-FunctionDecl @0x73be48 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:23> col:23 __u_char 'unsigned char'
  -BuiltinType @0x732b70 'unsigned char'
-FunctionDecl @0x73be08 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:28> col:28 __u_short 'unsigned short'
  -BuiltinType @0x732b90 'unsigned short'
-FunctionDecl @0x73bf28 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:22> col:22 __u_int 'unsigned int'
  -BuiltinType @0x732b80 'unsigned int'
-FunctionDecl @0x73bf98 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:27> col:27 __u_long 'unsigned long'
  -BuiltinType @0x732bd0 'unsigned long'
-FunctionDecl @0x73bc08 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:21> col:21 referenced __int8_t 'signed char'
  -BuiltinType @0x732ad0 'signed char'
-FunctionDecl @0x73bc78 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:23> col:23 referenced __uint8_t 'unsigned char'
  -BuiltinType @0x732b70 'unsigned char'
-FunctionDecl @0x73bc08 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:26> col:26 referenced __int16_t 'short'
  -BuiltinType @0x732af0 'short'
-FunctionDecl @0x73bc18 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:28> col:28 referenced __uint16_t 'unsigned short'
  -BuiltinType @0x732b90 'unsigned short'
-FunctionDecl @0x73bc18 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:28> col:28 referenced __int32_t 'int'
  -BuiltinType @0x732b10 'int'
-FunctionDecl @0x73bc18 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:22> col:22 referenced __uint32_t 'unsigned int'
  -BuiltinType @0x732bd0 'unsigned int'
-FunctionDecl @0x73bc28 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:25> col:25 referenced __int64_t 'long'
  -BuiltinType @0x732b30 'long'
-FunctionDecl @0x73bc18 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:27> col:27 referenced __uint64_t 'unsigned long'
  -BuiltinType @0x732bd0 'unsigned long'
-FunctionDecl @0x73bc08 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:18> col:18 __int_least8_t ' __int8_t': 'signed char'
  -TypeDecl @0x73bc370 ' __int8_t' sugar
    -TypeDecl @0x73bc08 ' __int8_t'
  -BuiltinType @0x732ad0 'signed char'
-FunctionDecl @0x73bc30 </usr/include/x86_64-linux-gnu/bits/types.h:31:1, col:19> col:19 __uint_least8_t ' __uint8_t': 'unsigned char'
  -TypeDecl @0x73bc370 ' __uint8_t' sugar
    -TypeDecl @0x73bc30 ' __uint8_t'
  -BuiltinType @0x732bd0 'unsigned char'

```

图 5: 语法分析结果

i. 主函数、函数体与变量声明

```

1 -FunctionDecl 0x1dcfe760 <main.c:12:1, line:41:1> line:12:5 main 'int
  ()' -CompoundStmt 0x1dcff998 <line:13:1, line:41:1>
2   | -DeclStmt 0x1dcfeab0 <line:14:5, col:22>
3   |   | -VarDecl 0x1dcfe818 <col:5, col:9> col:9 used a 'int'
4   |   | -VarDecl 0x1dcfe898 <col:5, col:12> col:12 used b 'int'
5   |   | -VarDecl 0x1dcfe918 <col:5, col:15> col:15 used i 'int'
6   |   | -VarDecl 0x1dcfe998 <col:5, col:18> col:18 used t 'int'
7   |   | -VarDecl 0x1dcfea18 <col:5, col:21> col:21 used n 'int'

```

最上层的 FunctionDecl 表示函数声明，节点名称 main 和类型 'int ()' 说明程序中定义了一个返回值为 int、不接收参数的 main 函数。它下面的 CompoundStmt 是复合语句节点，对应 main 函数的一对花括号，表示函数的整个函数体。

在函数体中，DeclStmt 表示一条变量声明语句，其下的五个 VarDecl 分别表示变量 a、b、i、t 和 n。每个节点后方的 'int' 表示变量的类型为整型；used 表示该变量在后续代码中确实被使用，而各个位置字段则用于记录变量声明在源程序中的位置。

ii. 赋值、输入输出与函数调用

```

1 | -BinaryOperator 0x1dcfeb08 <line:16:5, line:7:16> 'int' '='
2 |   | -DeclRefExpr 0x1dcfeac8 <line:16:5> 'int' lvalue Var 0x1dcfe818 'a' 'int'
3 |   | -IntegerLiteral 0x1dcfeae8 <line:7:16> 'int' 0 | -CallExpr 0x1dcff250 <line:20:5, col:19> 'int'|
4 |     | -ImplicitCastExpr 0x1dcff238 <col:5> | |
5 |       | -DeclRefExpr 0x1dcfebe8 <col:5> Function 0x1dcf3bc0 'scanf'
6 |       | -ImplicitCastExpr 0x1dcff280 <col:11> 'char *' <ArrayToPointerDecay>
7 |         | -StringLiteral 0x1dcfec48 <col:11> 'char[3]' lvalue ""
8 |           | -UnaryOperator 0x1dcff1a8 <col:17, col:18> 'int *' prefix '&'
9 |           | -DeclRefExpr 0x1dcfec68 <col:18> 'int' lvalue Var 0x1dcfea18 'n' 'int' | -CallExpr 0x1dcff340
10 |             <line:22:5, col:21> 'int' | -ImplicitCastExpr 0x1dcff328 <col:5> | |
11 |               | -DeclRefExpr 0x1dcff2b0 <col:5> Function 0x1dc3578 'printf'
12 |               | -ImplicitCastExpr 0x1dcff370 <col:12> 'char *' <ArrayToPointerDecay>

```

```

8 | | -StringLiteral 0x1dcff2d0 <col:12> 'char[4]' lvalue "|"
  | | -ImplicitCastExpr 0x1dcff3a0 <col:20> 'int' <LValueToRValue>
9 | | -DeclRefExpr 0x1dcff2f0 <col:20> 'int' lvalue Var 0x1dcfe818 'a' 'int'

```

BinaryOperator 表示二元运算。在 `a = 0` 的节点中，赋值号左侧的 DeclRefExpr 表示对变量 `a` 的引用，右侧的 IntegerLiteral 表示整数常量 0。需要注意的是，该常量的位置来自 `main.c` 第 7 行的宏定义，说明 `FIB_A0` 已经在预处理阶段被替换为 0，语法分析直接处理的是宏展开后的结果。

CallExpr 表示函数调用。`scanf` 调用中包含字符串字面量 `"%d"`、取地址运算符 `&` 以及变量 `n` 的引用，分别对应源程序中的格式字符串和 `&n`。同样，两个 `printf` 调用中包含字符串字面量 `"%d\n"`，并通过 DeclRefExpr 引用变量 `a` 或 `b`，从而确定了输出语句的语法结构。

ImplicitCastExpr 并没有直接写在源程序中，而是编译器根据类型规则自动加入的隐式转换节点。例如，函数名需要转换为函数指针，数组形式的字符串需要转换为字符指针，变量从存储位置读取数值时则会产生 LValueToRValue 转换。

iii. while 循环、循环体与宏函数展开

```

1 | |-WhileStmt 0x1dcff948 <line:29:5, line:38:5>
2 | | |-BinaryOperator 0x1dcff530 <line:29:12, col:16> 'int' '<'
3 | | | |-ImplicitCastExpr 0x1dcff500 <col:12> 'int' <LValueToRValue>
4 | | | | |-DeclRefExpr 0x1dcff4c0 <col:12> 'int' lvalue Var 0x1dcfe918 'i' | |
  | | | | -ImplicitCastExpr 0x1dcff518 <col:16> 'int' <LValueToRValue>
5 | | | | -DeclRefExpr 0x1dcff4e0 <col:16> 'int' lvalue Var 0x1dcfea18 'n' |
  | | | | -CompoundStmt 0x1dcff910 <line:30:5, line:38:5>
6 | | | | |-BinaryOperator 0x1dcff5a8 <line:31:9, col:13> 'int' '='
7 | | | | | |-DeclRefExpr 0x1dcff550 <col:9> 'int' lvalue Var 0x1dcfe998 't'
8 | | | | | -DeclRefExpr 0x1dcff570 <col:13> 'int' lvalue Var 0x1dcfe898 'b' | |-BinaryOperator
  | | | | | 0x1dcff678 <line:32:9, col:17> 'int' '=' | | |-DeclRefExpr 0x1dcff5c8 <col:9> 'int' lvalue Var
  | | | | | 0x1dcfe898 'b' | | -BinaryOperator 0x1dcff658 <col:13, col:17> 'int' '+'
9 | | | | | | |-DeclRefExpr 0x1dcff5e8 <col:13> 'int' lvalue Var 0x1dcfe818 'a'
10 | | | | | | -DeclRefExpr 0x1dcff608 <col:17> 'int' lvalue Var 0x1dcfe898 'b' |
  | | | | | | -BinaryOperator 0x1dcff8f0 <line:37:9, line:10:29> 'int' '='
11 | | | | | | | |-DeclRefExpr 0x1dcff818 <line:37:9> 'int' lvalue Var 0x1dcfe918 'i'
12 | | | | | | | -ParenExpr 0x1dcff8d0 <line:10:21, col:29> 'int' |
  | | | | | | | -BinaryOperator 0x1dcff8b0 <col:22, col:28> 'int' '+'
13 | | | | | | | | |-ParenExpr 0x1dcff858 <col:22, col:24> 'int' lvalue
14 | | | | | | | | | |-DeclRefExpr 0x1dcff838 <line:37:22> 'int' lvalue Var 'i' |
  | | | | | | | | | -IntegerLiteral 0x1dcff878 <line:10:28> 'int' 1
15 | -ReturnStmt 0x1dcff988 <line:40:5, col:12> -IntegerLiteral 0x1dcff968 <col:12> 'int' 0

```

WhileStmt 表示循环语句。它下面有两个主要子树：第一个是以 `<` 为运算符的 BinaryOperator，对应循环条件 `i < n`；第二个是 CompoundStmt，对应花括号包围的循环体。循环体中的循环变量更新、临时变量赋值和加法运算也都被构造成层次分明的 BinaryOperator 节点。

在 `i = FIB_NEXT(i)` 对应的节点中，没有出现名为 `FIB_NEXT` 的函数调用。预处理阶段已将 `FIB_NEXT(i)` 展开为 `((i) + 1)`，因此 AST 中表现为外侧的 ParenExpr、中间的加法 BinaryOperator、变量 `i` 所对应的 DeclRefExpr 以及整数常量 1。这一结构直接验证了函数宏展开确实发生在语法分析之前。

最后，ReturnStmt 表示返回语句，其下的 IntegerLiteral 0 对应 `return 0`。AST 中也没有出现 `#ifdef FIB_DEBUG` 控制的调试输出，因为本次编译没有定义 `FIB_DEBUG`，条件编译块在预处理阶段已被移除。

综上，Clang 将预处理后的 Token 序列成功组织成了具有层次关系的抽象语法树。函数定义、变量声明、赋值、函数调用、选择结构和循环结构都能在 AST 中找到对应的节点，说明语法分析器已经正确识别了源程序的语法结构，并为后续语义分析提供了基础。

语法错误检查验证

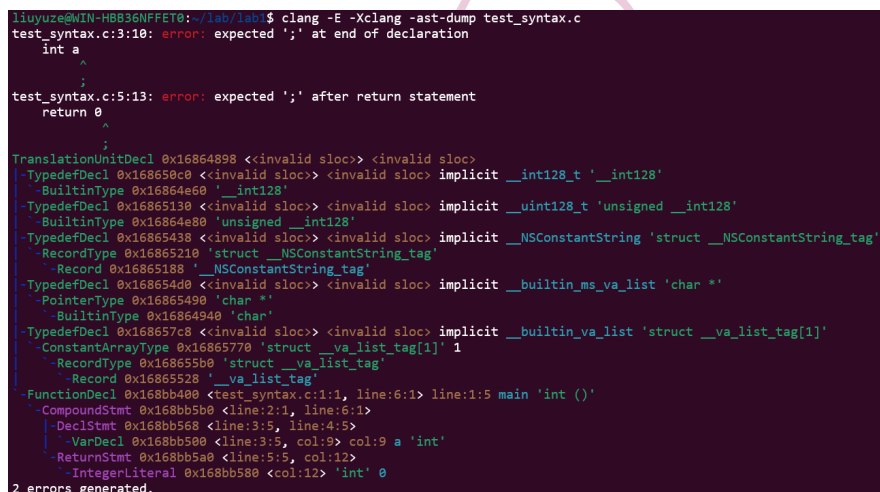
为了进一步验证语法分析器检查语法错误的功能，可以另外建立一个 `test_syntax.c`：

```
1 int main()
2 {
3     int a
4     a = 1 + ;
5     return 0
6 }
```

执行：

`clang -E -Xclang -ast-dump test_syntax.c`

报错结果如图 6 所示。



```
liuyuze@WIN-HBB36NFFET0:~/lab/lab1$ clang -E -Xclang -ast-dump test_syntax.c
test_syntax.c:3:10: error: expected ';' at end of declaration
    int a
    ^
test_syntax.c:5:13: error: expected ';' after return statement
    return 0
    ^
TranslationUnitDecl 0x16864898 <<invalid sloc>> <invalid sloc>
- TypedefDecl 0x168650c0 <<invalid sloc>> <invalid sloc> implicit __int128_t '__int128'
- BuiltinType 0x16864e60 '__int128'
- TypedefDecl 0x16865130 <<invalid sloc>> <invalid sloc> implicit __uint128_t 'unsigned __int128'
- BuiltinType 0x16864e80 'unsigned __int128'
- TypedefDecl 0x16865438 <<invalid sloc>> <invalid sloc> implicit __NSConstantString 'struct __NSConstantString_tag'
- RecordType 0x16865210 'struct __NSConstantString_tag'
- Record 0x16865188 '__NSConstantString_tag'
- TypedefDecl 0x168654d0 <<invalid sloc>> <invalid sloc> implicit __builtin_ms_va_list 'char *'
- PointerType 0x16865490 'char *'
- BuiltinType 0x168649d0 'char'
- TypedefDecl 0x16865728 <<invalid sloc>> <invalid sloc> implicit __builtin_va_list 'struct __va_list_tag[1]'
- ConstantArrayType 0x16865770 'struct __va_list_tag[1]' 1
- RecordType 0x168655b0 'struct __va_list_tag'
- Record 0x16865528 '__va_list_tag'
- FunctionDecl 0x1686b400 <test_syntax.c:1:1, line:6:1> line:1:5 main 'int ()'
- CompoundStmt 0x1686b5b0 <line:2:1, line:6:1>
- DeclStmt 0x1686b568 <line:3:5, line:4:5>
- VarDecl 0x1686b500 <line:3:5, col:9> col:9 a 'int'
- ReturnStmt 0x1686b5a0 <line:5:5, col:12>
- IntegerLiteral 0x1686b500 <col:12> 'int' 0
2 errors generated.
```

图 6: 语法错误检查结果

可以观察到，编译器能够指出变量声明后缺少分号、加法表达式缺少右操作数以及 `return` 语句后缺少分号等语法问题。由此说明，语法分析不仅能构造 AST，还能够发现不符合 C 语法规则的代码。

(3) 语义分析

语义分析的主要功能如下：

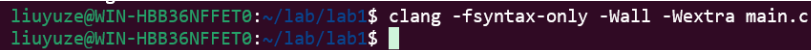
- **名称解析：**将变量名、函数名等标识符与抽象语法树中对应的声明建立联系，明确每个标识符引用的是哪一个变量或函数。
- **符号表构建与查询：**记录程序中变量和函数的名称、类型、作用域等信息，并在使用标识符时通过符号表查找其声明。
- **类型检查：**检查赋值、算术运算、关系运算和函数调用中的类型是否兼容，例如赋值号两侧应具有相容的类型，函数实参与形参匹配。

- **作用域检查**：判断标识符在当前作用域内是否可见，确保局部变量只能在其作用域中使用，避免访问未声明的名称。
- **隐式类型转换**：根据 C 语言的类型规则自动插入必要的转换，例如函数名转换为函数指针、数组转换为指针以及左值转换为右值。
- **控制流与数据流分析**：分析程序的执行路径和数据传递过程，用于发现未初始化变量、不可达代码等问题，并为后续代码优化提供依据。
- **错误检测与报告**：在发现类型不匹配、符号未声明或作用域错误时，给出错误所在位置和原因，阻止不满足语义规则的程序继续生成目标代码。

使用如下命令对 main.c 进行语义检查：

```
clang -fsyntax-only -Wall -Wextra main.c
```

如果终端没有输出错误信息，说明程序中的变量、函数、类型和表达式均通过检查。检查结果如图 7 所示。



```
liuyuze@WIN-HBB36NFFET0:~/lab/lab1$ clang -fsyntax-only -Wall -Wextra main.c
liuyuze@WIN-HBB36NFFET0:~/lab/lab1$
```

图 7: 语义检查成功结果

语义错误检查验证

创建test_semantic.c 如下：

```
1  struct A { int x; };
2  struct B { int y; };
3
4  int f(int x)
5  {
6      return x;
7  }
8
9  void g(struct A value)
10 {
11     (void)value;
12 }
13
14 int conflict(void)
15 {
16     return 0;
17 }
18
19 int conflict(void)
20 {
21     return 1;
22 }
```

```
23
24 int main(void)
25 {
26     int a;
27     struct B b;
28
29     b = a;
30     f();
31     g(a);
32     unknown = 1;
33
34     int duplicate = 0;
35     int duplicate = 1;
36
37     break;
38     return 0;
39 }
```

执行：

```
clang -fsyntax-only test_semantic.c
```

检查结果如图 8 所示。

```
liuyuze@WIN-HBB36NFFET0:~/lab/lab1$ clang -fsyntax-only test_semantic.c
test_semantic.c:19:5: error: redefinition of 'conflict'
int conflict(void)
^
test_semantic.c:14:5: note: previous definition is here
int conflict(void)
^
test_semantic.c:29:7: error: assigning to 'struct B' from incompatible type 'int'
    b = a;
    ^
test_semantic.c:30:7: error: too few arguments to function call, single argument 'x' was not specified
    f();
    ^
test_semantic.c:4:5: note: 'f' declared here
int f(int x)
^
test_semantic.c:31:7: error: passing 'int' to parameter of incompatible type 'struct A'
    g(a);
    ^
test_semantic.c:9:17: note: passing argument to parameter 'value' here
void g(struct A value)
^
test_semantic.c:32:5: error: use of undeclared identifier 'unknown'
    unknown = 1;
    ^
test_semantic.c:35:9: error: redefinition of 'duplicate'
    int duplicate = 1;
    ^
test_semantic.c:34:9: note: previous definition is here
    int duplicate = 0;
    ^
test_semantic.c:37:5: error: 'break' statement not in loop or switch statement
    break;
    ^
7 errors generated.
```

图 8: 语义错误检查结果

综上，Clang 在语法分析生成的 AST 基础上完成了变量与函数的名称解析、表达式类型检查以及函数指针、数组指针和左值到右值等隐式转换。对于不符合语义规则的程序，编译器能够通过符号表和控制流检查发现未声明标识符、类型不兼容、函数调用错误以及符号重复定义等问题，为后续中间代码生成提供正确的语义信息。

(4) 中间代码生成

中间代码生成的主要功能是：在语义分析确认源程序合法之后，将抽象语法树转换为一种与具体目标机器关系较小的中间表示（Intermediate Representation, IR）。中间表示既要保留源程序中的运算、控制流和数据流信息，又要便于后续进行代码优化和目标代码生成。本实验所使用的 Clang/LLVM 编译器将 C 源程序转换为 LLVM IR，其文本形式保存在 `main.ll` 文件中。

中间代码生成的主要功能可以分点说明如下:

- **降低语法树层次**: 将具有嵌套结构的表达式和语句转换为一条条顺序明确的指令, 使程序结构更接近机器能够处理的形式。
- **生成中间表示**: 把赋值、算术运算、函数调用和返回等操作转换为 LLVM IR 指令, 例如 `store`、`load`、`add`、`call` 和 `ret`。
- **构造控制流图**: 把条件判断和循环转换为基本块以及基本块之间的跳转关系, 使用 `br`、`icmp` 等指令表示程序执行路径。
- **保存程序数据信息**: 为局部变量分配栈空间, 生成相应的 `alloca`、`load` 和 `store` 指令, 并通过虚拟寄存器暂时保存中间计算结果。
- **为后续阶段提供输入**: 生成的 LLVM IR 与目标机器暂时解耦, 既可用于代码优化, 也可继续生成 x86、ARM 等不同平台的目标代码。

本次实验使用如下命令生成 LLVM IR:

```
clang -S -emit-llvm main.c -o main.ll
```

其中, `-S` 表示输出文本形式的代码, `-emit-llvm` 表示生成 LLVM IR, `-o main.ll` 表示将结果保存到 `main.ll` 文件。命令执行结果如图 9 所示。

[illegible]

图 9: 生成 main.ll 中间代码

下面只选取具有代表性的部分进行分析：

i. 模块信息与全局字符串

```

1 ; ModuleID = '/home/liuyuze/lab/lab1/main.c'
2 source_filename = "/home/liuyuze/lab/lab1/main.c"
3 target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8
   :16:32:64-S128"
4 target triple = "x86_64-pc-linux-gnu"
5
6 @.str = private unnamed_addr constant [3 x i8] c"%d\00", align 1
7 @.str.1 = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

```

文件开头给出了当前模块的编号、源文件名、目标数据布局和目标三元组。目标三元组 `x86_64-pc-linux-gnu` 说明本次生成的是面向 64 位 x86 Linux 平台的 IR。

符号 `@.str` 和 `@.str.1` 是两个全局常量，其类型分别为 `[3 x i8]` 和 `[4 x i8]`，内容对应 `"%d"` 和 `"%d\n"`。其中 `\00` 表示字符串结尾的空字符，`\0A` 表示换行符。程序中的格式字符串没有被直接嵌入每条调用指令，而是被提取为全局变量，由函数调用时传入。

ii. 主函数和局部变量存储

```

1 ; Function Attrs: noinline nounwind optnone uwtable
2 define dso_local i32 @main() #0 {
3     %1 = alloca i32, align 4
4     %2 = alloca i32, align 4
5     %3 = alloca i32, align 4
6     %4 = alloca i32, align 4
7     %5 = alloca i32, align 4
8     %6 = alloca i32, align 4
9     store i32 0, i32* %1, align 4
10    store i32 0, i32* %2, align 4
11    store i32 1, i32* %3, align 4
12    store i32 1, i32* %4, align 4

```

`define dso_local i32 @main()` 表示定义一个返回值为 `i32` 的 `main` 函数。函数上方还给出了 `noinline`、`nounwind`、`optnone` 和 `uwtable` 等属性。其中 `optnone` 表示当前为未优化编译，便于直接观察编译器生成的原始中间代码。

源程序中的五个 `int` 变量在 LLVM IR 中对应 `i32` 类型。六条 `alloca` 指令用于在栈上分配整型存储空间，`align 4` 表示按照 4 字节对齐。随后使用 `store` 将变量初值写入对应的存储位置，例如变量 `a`、`b` 和 `i` 被分别赋值为 0、1 和 1。

IR 中没有出现 `FIB_A0`、`FIB_B0`、`FIB_IO` 和 `FIB_NEXT`，说明宏替换已经在预处理阶段完成。源程序中的宏常量已经变成整数常量，函数宏 `FIB_NEXT(i)` 也已经变成普通的加法运算。

iii. 输入输出函数调用

```

1 %7 = call i32 (@__isoc99_scanf(
2     i8* noundef getelementptr inbounds ([3 x i8], [3 x i8]* @.str, i64 0, i64 0),
3     i32* noundef %6)
4 %8 = load i32, i32* %2, align 4
5 %9 = call i32 (@printf(
6     i8* noundef getelementptr inbounds ([4 x i8], [4 x i8]* @.str.1, i64 0, i64 0)

```

```
7 | i32 noundef %8)
```

`__isoc99_scanf` 和 `printf` 都被生成外部函数调用。调用 `scanf` 时, `getelementptr` 用于取得全局字符串 `@.str` 的首地址, `%6` 则是变量 `n` 的存储地址, 因此它能够接收输入数据。

`printf` 调用前先使用 `load` 从变量 `a` 对应的存储位置 `%2` 中读取整数值, 再将读取结果 `%8` 作为参数传入。这里体现了 LLVM IR 中“存储位置”和“寄存器中的值”之间的区别: `alloca` 和 `store` 负责保存变量, `load` 负责在参与计算或输出时读出变量值。

iv. while 循环的基本块与控制流

```
1 | br label %12
2 |
3 | 12:                                ; preds = %16, %0
4 | %13 = load i32, i32* %4, align 4
5 | %14 = load i32, i32* %6, align 4
6 | %15 = icmp slt i32 %13, %14
7 | br i1 %15, label %16, label %26
8 |
9 | 16:                                ; preds = %12
10 | %17 = load i32, i32* %3, align 4
11 | store i32 %17, i32* %5, align 4
12 | %18 = load i32, i32* %2, align 4
13 | %19 = load i32, i32* %3, align 4
14 | %20 = add nsw i32 %18, %19
15 | store i32 %20, i32* %3, align 4
16 | %21 = load i32, i32* %3, align 4
17 | %22 = call i32 @i8*, ... @printf(
18 |     i8* noundef getelementptr @inbounds ([4 x i8], [4 x i8]* @.str.1, i64 0, i64 0)
19 |     ,
20 |     i32 noundef %21)
21 | %23 = load i32, i32* %5, align 4
22 | store i32 %23, i32* %2, align 4
23 | %24 = load i32, i32* %4, align 4
24 | %25 = add nsw i32 %24, 1
25 | store i32 %25, i32* %4, align 4
26 | br label %12, !llvm.loop !6
27 | 26:                                ; preds = %12
28 | ret i32 0
```

基本块 12 是循环条件块。它先使用两条 `load` 分别读取变量 `i` 和 `n` 的值, 再用 `icmp slt` 判断 `i` 是否小于 `n`。 `icmp` 用于整数比较, `slt` 表示有符号小于比较; 比较结果保存在布尔值 `%15` 中, 随后由 `br i1` 指令决定程序的执行方向。

当条件成立时, 程序跳转到基本块 16, 也就是循环体。循环体依次完成保存旧变量、计算 `a + b`、调用 `printf` 输出结果、更新变量 `a` 以及执行 `i = i + 1` 等操作。 `add nsw` 表示带符号整数加法, 并允许编译器假定运算不发生有符号溢出。循环体最后通过 `br label %12` 跳回条件块, 形成循环结构。

当变量 `i` 不再小于 `n` 时, `br i1` 会选择跳转到基本块 26。该基本块使用 `ret i32 0` 返回整数 0, 对应源程序中的 `return 0;`。因此, `while` 循环在 LLVM IR 中被明确表示为条件块、循环体和退出块这三个基本块之间的控制流关系。

综上，中间代码生成阶段将经过词法、语法和语义分析的 C 程序转换为 LLVM IR。源程序中的变量、表达式、函数调用和循环控制结构分别被表示为栈空间、load 和 store、call、icmp 以及基本块跳转。生成的 IR 与具体目标机器解耦，为后续代码优化和目标代码生成提供了统一输入。

(5) 代码优化

代码优化是在不改变程序运行结果的前提下，对 LLVM IR 进行等价变换。优化器会尝试减少冗余的栈空间访问、传播常量、简化控制流，并为循环生成更紧凑的指令。

实验一：观察 opt 对 LLVM IR 的处理结果

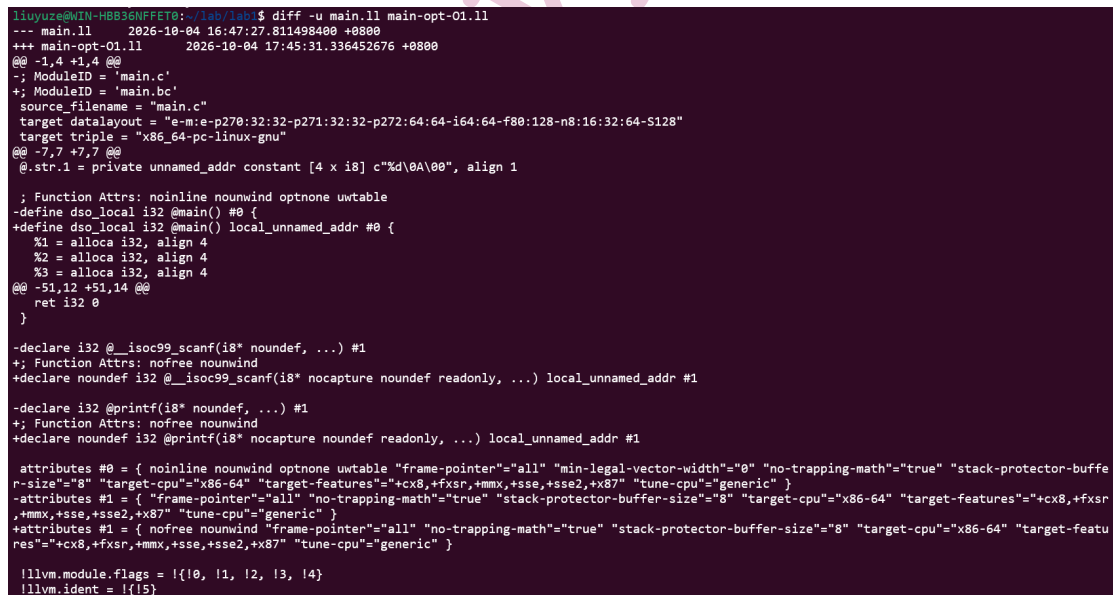
中间代码生成阶段得到的 main.ll 默认由未优化的 Clang 生成，函数上带有 optnone 属性。先将文本 IR 转换为 LLVM 位码，再分别执行 O1、O2 和 O3 优化：

```
llvm-as main.ll -o main.bc
opt -S -O1 main.bc -o main-opt-01.ll
opt -S -O2 main.bc -o main-opt-02.ll
opt -S -O3 main.bc -o main-opt-03.ll
```

接着比较优化前后的差异：

```
diff -u main.ll main-opt-01.ll
```

终端会显示两份 IR 的差异，其中 - 开头的是未优化文件中的内容，+ 开头的是优化后新增或修改的内容。



```
liuyuze@WIN-HBB3GNFFET0:~/lab1$ diff -u main.ll main-opt-01.ll
--- main.ll      2026-10-04 16:47:27.811498400 +0800
+++ main-opt-01.ll  2026-10-04 17:45:31.336452676 +0800
@@ -1,4 +1,4 @@
-; ModuleID = 'main.c'
+; ModuleID = 'main.bc'
+  source_filename = "main.c"
  target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
  target triple = "x86_64-pc-linux-gnu"
@@ -7,7 +7,7 @@
 @.str.1 = private unnamed_addr constant [4 x i8] c"%d\\n", align 1

; Function Attrs: noinline nounwind optnone uwtable
-define dso_local i32 @main() #0 {
+define dso_local i32 @main() local_unnamed_addr #0 {
    %1 = alloca i32, align 4
    %2 = alloca i32, align 4
    %3 = alloca i32, align 4
@@ -51,12 +51,14 @@
    ret i32 0
}

-declare i32 @__isoc99_scanf(i8* noundef, ...) #1
+; Function Attrs: nofree nounwind
+declare noundef i32 @__isoc99_scanf(i8* nocapture noundef readonly, ...) local_unnamed_addr #1

-declare i32 @printf(i8* noundef, ...) #1
+; Function Attrs: nofree nounwind
+declare noundef i32 @printf(i8* nocapture noundef readonly, ...) local_unnamed_addr #1

  attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
  attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
+attributes #1 = { nofree nounwind "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

  !llvm.module.flags = !{!0, !1, !2, !3, !4}
  !llvm.ident = !{!5}
```

图 10: 优化前后的 LLVM IR 差异

从差异可以看到，ModuleID 由 main.c 变为 main.bc，这是文本 IR 转换为位码后再次输出的结果。main 的定义增加了 local_unnamed_addr 属性，scanf 和 printf 的声明增加了 noundef、nocapture、readonly 和 local_unnamed_addr 等属性，属性组中同时增加了 nofree 和 nounwind。这些信息向优化器说明函数的地址是否重要、参数指向的内存是否会被修改以及函数是否会抛出异常，从而为后续优化提供依据。

需要注意，此时 main 仍然带有未优化编译产生的 optnone 属性：

```
; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() local_unnamed_addr #0 {
```

因此，opt 不能深入优化 main 的函数体，只能修改函数和库声明的属性。差异中出现的主要变化是属性和地址标记，而不是 while 循环被整体替换。这个结果说明优化 pass 会受到函数属性的限制，也为下一步从源码开始使用不同优化级别生成 IR 提供了对照。

实验二：比较不同优化级别生成的 IR

为了观察真正的循环优化效果，需要让 Clang 在生成 IR 时直接启用优化。依次执行：

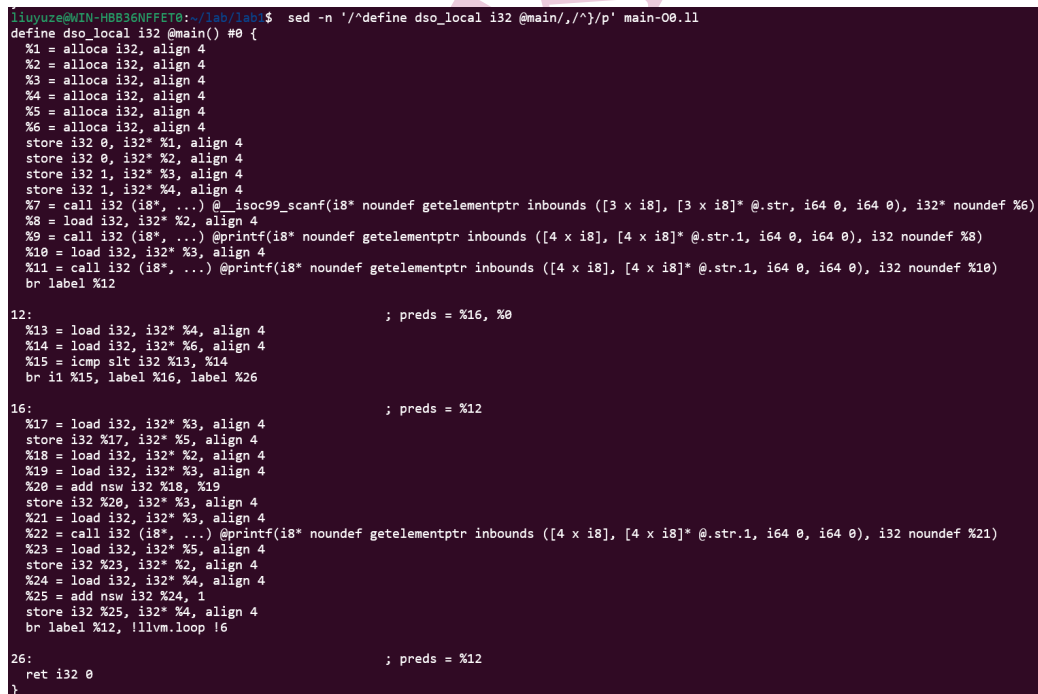
```
clang -S -emit-llvm -O0 main.c -o main-00.ll
clang -S -emit-llvm -O1 main.c -o main-01.ll
clang -S -emit-llvm -O2 main.c -o main-02.ll
clang -S -emit-llvm -O3 main.c -o main-03.ll
```

```
wc -l main-00.ll main-01.ll main-02.ll main-03.ll
```

分别查看未优化和 O1 优化后的 main 函数：

```
sed -n '/^define dso_local i32 @main/,/^}/p' main-00.ll
sed -n '/^define dso_local i32 @main/,/^}/p' main-01.ll
```

两张截图分别保存为 code-optimization-o0.png 和 code-optimization-o1.png。



```
liuyuze@MIN-HBB36NFFET0:~/lab/lab1$ sed -n '/^define dso_local i32 @main/,/^}/p' main-00.ll
define dso_local i32 @main() #0 {
    %1 = alloca i32, align 4
    %2 = alloca i32, align 4
    %3 = alloca i32, align 4
    %4 = alloca i32, align 4
    %5 = alloca i32, align 4
    %6 = alloca i32, align 4
    store i32 0, i32* %1, align 4
    store i32 0, i32* %2, align 4
    store i32 1, i32* %3, align 4
    store i32 1, i32* %4, align 4
    %7 = call i32 (@__isoc99_scanf(i8* noundef getelementptr inbounds ([3 x i8], [3 x i8]* @.str, i64 0, i64 0), i32* noundef %6)
    %8 = load i32, i32* %2, align 4
    %9 = call i32 (@__isoc99_printf(i8* noundef getelementptr inbounds ([4 x i8], [4 x i8]* @.str.1, i64 0, i64 0), i32 noundef %8)
    %10 = load i32, i32* %3, align 4
    %11 = call i32 (@__isoc99_printf(i8* noundef getelementptr inbounds ([4 x i8], [4 x i8]* @.str.1, i64 0, i64 0), i32 noundef %10)
    br label %12

12:                                ; preds = %16, %0
    %13 = load i32, i32* %4, align 4
    %14 = load i32, i32* %6, align 4
    %15 = icmp slt i32 %13, %14
    br i1 %15, label %16, label %26

16:                                ; preds = %12
    %17 = load i32, i32* %3, align 4
    store i32 %17, i32* %5, align 4
    %18 = load i32, i32* %2, align 4
    %19 = load i32, i32* %3, align 4
    %20 = add nsw i32 %18, %19
    store i32 %20, i32* %3, align 4
    %21 = load i32, i32* %3, align 4
    %22 = call i32 (@__isoc99_printf(i8* noundef getelementptr inbounds ([4 x i8], [4 x i8]* @.str.1, i64 0, i64 0), i32 noundef %21)
    %23 = load i32, i32* %5, align 4
    store i32 %23, i32* %2, align 4
    %24 = load i32, i32* %4, align 4
    %25 = add nsw i32 %24, 1
    store i32 %25, i32* %4, align 4
    br label %12, !llvm.loop !6

26:                                ; preds = %12
    ret i32 0
}
```

图 11: -O0 生成的主函数 IR

```

liuyuze@WIN-HBB36NFFET0:~/lab/lab1$ sed -n '/^define dso_local i32 @main/,/^}/p' main-01.ll
define dso_local i32 @main() local_unnamed_addr #0 {
    %1 = alloca i32, align 4
    %2 = bitcast i32* %1 to i8*
    call void @llvm.lifetime.start.p0i8(i64 4, i8* nonnull %2) #3
    %3 = call i32 (i8*, ...) @__isoc99_scanf(i8* noundef getelementptr inbounds ([3 x i8], [3 x i8]* @.str, i64 0, i64 0), i32* noundef nonnull %1)
    %4 = call i32 (i8*, ...) @printf(i8* noundef nonnull dereferenceable(1) getelementptr inbounds ([4 x i8], [4 x i8]* @.str.1, i64 0, i64 0), i32 noundef 0)
    %5 = call i32 (i8*, ...) @printf(i8* noundef nonnull dereferenceable(1) getelementptr inbounds ([4 x i8], [4 x i8]* @.str.1, i64 0, i64 0), i32 noundef 1)
    %6 = load i32, i32* %1, align 4, !tbaa !5
    %7 = icmp sgt i32 %6, 1
    br i1 %7, label %8, label %17

8:
    %9 = phi i32 [ %11, %8 ], [ 0, %0 ]
    %10 = phi i32 [ %14, %8 ], [ 1, %0 ]
    %11 = phi i32 [ %12, %8 ], [ 1, %0 ]
    %12 = add nsw i32 %9, %11
    %13 = call i32 (i8*, ...) @printf(i8* noundef nonnull dereferenceable(1) getelementptr inbounds ([4 x i8], [4 x i8]* @.str.1, i64 0, i64 0), i32 noundef %12)
    %14 = add nuw nsw i32 %10, 1
    %15 = load i32, i32* %1, align 4, !tbaa !5
    %16 = icmp slt i32 %14, %15
    br i1 %16, label %8, label %17, !llvm.loop !9

17:
    call void @llvm.lifetime.end.p0i8(i64 4, i8* nonnull %2) #3
    ret i32 0
}

```

图 12: -O1 优化后的主函数 IR

未优化的 main-00.ll 为每个局部变量分配栈空间，反复使用 `alloca`、`load` 和 `store` 完成变量读写，例如 `a`、`b`、`i` 和 `t` 都通过栈位置保存。O1 优化后，这些变量的栈存储基本被消除：变量 `n` 仍需保留一个 `alloca`，因为它要接收 `scanf` 写入的地址。函数入口直接使用常量 0 和 1 完成前两次输出，循环中的变量状态改用 `phi` 指令保存。循环初值 `i = 1` 和条件 `i < n` 还被合并为入口处的 `n > 1` 判断：

```

%1 = alloca i32, align 4
%2 = bitcast i32* %1 to i8*
call void @llvm.lifetime.start.p0i8(i64 4, i8* nonnull %2) #3
%3 = call i32 (i8*, ...) @__isoc99_scanf(
    i8* noundef getelementptr inbounds ([3 x i8], [3 x i8]* @.str,
    i64 0, i64 0), i32* noundef nonnull %1)
%4 = call i32 (i8*, ...) @printf(
    i8* noundef nonnull dereferenceable(1)
    getelementptr inbounds ([4 x i8], [4 x i8]* @.str.1,
    i64 0, i64 0), i32 noundef 0)
%5 = call i32 (i8*, ...) @printf(
    i8* noundef nonnull dereferenceable(1)
    getelementptr inbounds ([4 x i8], [4 x i8]* @.str.1,
    i64 0, i64 0), i32 noundef 1)
%6 = load i32, i32* %1, align 4, !tbaa !5
%7 = icmp sgt i32 %6, 1
br i1 %7, label %8, label %17

8:
    %9 = phi i32 [ %11, %8 ], [ 0, %0 ]
    %10 = phi i32 [ %14, %8 ], [ 1, %0 ]
    %11 = phi i32 [ %12, %8 ], [ 1, %0 ]
    %12 = add nsw i32 %9, %11
    %13 = call i32 (i8*, ...) @printf(
    i8* noundef nonnull dereferenceable(1)
    getelementptr inbounds ([4 x i8], [4 x i8]* @.str.1,

```

```

        i64 0, i64 0),
        i32 noundef %12)
%14 = add nuw nsw i32 %10, 1
%15 = load i32, i32* %1, align 4, !tbaa !5
%16 = icmp slt i32 %14, %15
br i1 %16, label %8, label %17, !llvm.loop !9

```

17:

```

call void @llvm.lifetime.end.p0i8(i64 4, i8* nonnull %2) #3
ret i32 0

```

其中,phi 根据前驱基本块选择循环变量当前值;add nsw 对应斐波那契数列加法;add nuw nsw 对应 FIB_NEXT(i) 展开后的自增操作;llvm.lifetime.start 和 llvm.lifetime.end 只用于标记局部存储对象的生命周期,不参与输入输出计算。可以看到, O1 已经完成了大部分栈存储消除、常量替换和循环状态合并,而不是只修改函数的属性。

实验三：比较不同优化级别下的运行时间

为了比较优化效果,新建 bench.c:

```

1  #include <stdio.h>
2  #include <stdint.h>
3  #include <time.h>
4
5  static volatile uint64_t sink;
6
7  int main(void)
8  {
9      int n, trials;
10
11     if (scanf("%d%d", &n, &trials) != 2 || n < 1 || trials < 1)
12         return 1;
13
14     clock_t start = clock();
15
16     for (int k = 0; k < trials; ++k)
17     {
18         int a = 0, b = 1, i = 1, t;
19
20         while (i < n)
21         {
22             t = b;
23             b = a + b;
24             a = t;
25             i = i + 1;
26         }
27
28         sink += (uint64_t)(unsigned int)b;

```

```

29     }
30
31     clock_t end = clock();
32     double total = (double)(end - start) / CLOCKS_PER_SEC;
33
34     printf("n=%d trials=%d total=%.6f average=%.9f sink=%llu\n",
35           n, trials, total, total / trials,
36           (unsigned long long)sink);
37
38     return 0;
39 }

```

使用相同源文件分别生成四个不同优化级别的可执行文件：

```

clang -O0 bench.c -o bench-00
clang -O1 bench.c -o bench-01
clang -O2 bench.c -o bench-02
clang -O3 bench.c -o bench-03

```

```
ls -lh bench-00 bench-01 bench-02 bench-03
```

使用相同的输入运行四个程序：

```

printf "40 1000000\n" | ./bench-00
printf "40 1000000\n" | ./bench-01
printf "40 1000000\n" | ./bench-02
printf "40 1000000\n" | ./bench-03

```

四次运行的输出如图所示。每次输出的 `sink` 均为 1023341550000000，说明不同优化级别计算得到的斐波那契结果一致，优化没有改变程序的运行结果。

```

liuyuze@WIN-HBB36NFFET0:~/lab/lab1$ printf "40 1000000\n" | ./bench-00
n=40 trials=1000000 total=0.045488 average=0.000000045 sink=1023341550000000
liuyuze@WIN-HBB36NFFET0:~/lab/lab1$ printf "40 1000000\n" | ./bench-01
n=40 trials=1000000 total=0.012994 average=0.000000013 sink=1023341550000000
liuyuze@WIN-HBB36NFFET0:~/lab/lab1$ printf "40 1000000\n" | ./bench-02
n=40 trials=1000000 total=0.008233 average=0.000000008 sink=1023341550000000
liuyuze@WIN-HBB36NFFET0:~/lab/lab1$ printf "40 1000000\n" | ./bench-03
n=40 trials=1000000 total=0.005803 average=0.000000006 sink=1023341550000000

```

图 13: 不同优化级别的运行时间

从结果可以看出，O1 的运行时间约为 O0 的 0.29 倍，加速约 3.50 倍；O2 约为 O0 的 0.18 倍，加速约 5.53 倍；O3 约为 O0 的 0.13 倍，加速约 7.84 倍。性能变化与前面的 IR 分析相互对应：O0 反复使用栈空间完成变量读写，O1 开始使用寄存器状态的 phi 合并、常量替换和循环简化，O2 与 O3 在此基础上继续精简和调度循环指令。

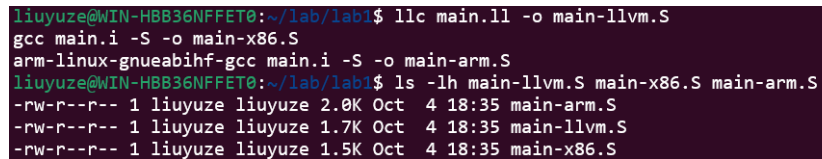
(6) 代码生成

代码生成以中间表示形式为输入，将其映射为目标机器的汇编代码。使用 `llc` 从 `main.ll` 生成目标代码，再使用 `clang -S` 生成 x86 汇编代码：

代码生成以中间表示形式为输入，将其映射为目标机器的汇编代码。本次实验使用 `llc` 从 `main.ll` 生成目标代码，再分别生成 `x86` 和 `ARM` 汇编代码：

```
llc main.ll -o main-llvm.S
gcc main.i -S -o main-x86.S
arm-linux-gnueabi-gcc main.i -S -o main-arm.S
ls -lh main-llvm.S main-x86.S main-arm.S
```

生成结果如图 14 所示。



```
liuyuze@WIN-HB836NFFET0:~/lab/lab1$ llc main.ll -o main-llvm.S
gcc main.i -S -o main-x86.S
arm-linux-gnueabi-gcc main.i -S -o main-arm.S
liuyuze@WIN-HB836NFFET0:~/lab/lab1$ ls -lh main-llvm.S main-x86.S main-arm.S
-rw-r--r-- 1 liuyuze liuyuze 2.0K Oct  4 18:35 main-arm.S
-rw-r--r-- 1 liuyuze liuyuze 1.7K Oct  4 18:35 main-llvm.S
-rw-r--r-- 1 liuyuze liuyuze 1.5K Oct  4 18:35 main-x86.S
```

图 14: 代码生成

4. 汇编器

汇编过程实际上把汇编语言程序代码翻译成目标机器指令的过程，其最终生成的是可重定位的机器代码。汇编器会将汇编程序中的伪指令翻译成等价的机器语言指令，将程序中的分支和数据传输指令中用到的标号都放到一个符号表中，我们通过查表可以生成对应指令的二进制。

我们使用汇编文件 `fib.s` 进行汇编，并通过 `objdump -d` 对 `fib.o` 进行反汇编，结果如图 15 所示。可以看到，`main` 函数的地址从 0 开始，说明尚未链接。

```

0000000000000000 <main>:
0: 7179          addi    sp,sp,-48
2: f406          sd      ra,40(sp)
4: f022          sd      s0,32(sp)
6: 1800          addi    s0,sp,48
8: 00000797      auipc   a5,0x0
c: 0007b783      ld       a5,0(a5) # 8 <main+0x8>
10: 6398          ld       a4,0(a5)
12: fee43423      sd      a4,-24(s0)
16: 4701          li       a4,0
18: fc042c23      sw      zero,-40(s0)
1c: 4785          li       a5,1
1e: fcf42e23      sw      a5,-36(s0)
22: 4785          li       a5,1
24: fef42023      sw      a5,-32(s0)
28: fd440793      addi    a5,s0,-44
2c: 85be          mv      a1,a5
2e: 00000517      auipc   a0,0x0
32: 00053503      ld      a0,0(a0) # 2e <main+0x2e>
36: 00000097      auipc   ra,0x0
3a: 000080e7      jalr    ra # 36 <main+0x36>
3e: fd842783      lw      a5,-40(s0)
42: 85be          mv      a1,a5
44: 00000517      auipc   a0,0x0
48: 00053503      ld      a0,0(a0) # 44 <main+0x44>
4c: 00000097      auipc   ra,0x0
50: 000080e7      jalr    ra # 4c <main+0x4c>
54: 87aa          mv      a5,a0
56: 00000597      auipc   a1,0x0
5a: 0005b583      ld      a1,0(a1) # 56 <main+0x56>
5e: 853e          mv      a0,a5
60: 00000097      auipc   ra,0x0
64: 000080e7      jalr    ra # 60 <main+0x60>
68: fdc42783      lw      a5,-36(s0)
6c: 85be          mv      a1,a5
6e: 00000517      auipc   a0,0x0
72: 00053503      ld      a0,0(a0) # 6e <main+0x6e>
76: 00000097      auipc   ra,0x0
7a: 000080e7      jalr    ra # 76 <main+0x76>
7e: 87aa          mv      a5,a0
80: 00000597      auipc   a1,0x0
84: 0005b583      ld      a1,0(a1) # 80 <main+0x80>
88: 853e          mv      a0,a5
8a: 00000097      auipc   ra,0x0
8e: 000080e7      jalr    ra # 8a <main+0x8a>
92: a899          j       e8 <.L2>

```

图 15: fib.o 反汇编结果

nm 命令显示的符号表如图 16 所示，T main 表示已定义的全局代码符号，U 开头的表示未定义、需要链接时解析的外部符号。

```

sun@sun-virtual-machine:~/Desktop$ riscv64-linux-gnu-nm fib.o
                 U __cxa_atexit
                 U __dso_handle
000000000000018e t _GLOBAL__sub_I_main
0000000000000000 T main
                 U __stack_chk_fail
                 U __stack_chk_guard
0000000000000120 t _Z41__static_initialization_and_destruction_0ii
                 U _ZNSirsERi
                 U _ZNSolsEi
                 U _ZNSolsEPFRSoS_E
                 U _ZNSt8ios_base4InitC1Ev
                 U _ZNSt8ios_base4InitD1Ev
                 U _ZSt3cin
                 U _ZSt4cout
                 U _ZSt4endlC5t11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_
0000000000000000 b _ZStL8__ioinit

```

图 16: fib.o 符号表

objdump -r 显示的重定位表如图 17 所示，记录了 .text 中需要在链接时修正的位置

```

sun@sun-virtual-machine:~/Desktop$ riscv64-linux-gnu-objdump -h fib.o

fib.o: 文件格式 elf64-littleriscv

节:
Idx Name          Size      VMA               LMA               File off  Algn
---
 0 .text          000001ae 0000000000000000 0000000000000000 00000040 2**1
    CONTENTS, ALLOC, LOAD, RELOC, READONLY, CODE
 1 .data           00000000 0000000000000000 0000000000000000 000001ee 2**0
    CONTENTS, ALLOC, LOAD, DATA
 2 .bss            00000001 0000000000000000 0000000000000000 000001f0 2**3
    ALLOC
 3 .init_array     00000008 0000000000000000 0000000000000000 000001f0 2**3
    CONTENTS, ALLOC, LOAD, RELOC, DATA
 4 .comment        0000002c 0000000000000000 0000000000000000 000001f8 2**0
    CONTENTS, READONLY
 5 .note.GNU-stack 00000000 0000000000000000 0000000000000000 00000224 2**0
    CONTENTS, READONLY
 6 .eh_frame       00000090 0000000000000000 0000000000000000 00000228 2**3
    CONTENTS, ALLOC, LOAD, RELOC, READONLY, DATA
 7 .riscv.attributes 00000033 0000000000000000 0000000000000000 000002b8 2**0
    CONTENTS, READONLY

```

图 17: fib.o 重定位表

综上，汇编器完成了从汇编到可重定位目标文件的翻译，并留下符号表和重定位信息，供链接器使用。

5. 链接器

由汇编程序生成的目标文件不能够直接执行。大型程序常常被分成多个部分进行编译，因此，可重定位的机器代码有必要和其他可重定位的目标文件以及库文件链接到一起，最终形成真正在机器上运行的代码。进而链接器对该机器代码进行执行生成可执行文件。

我们使用如下命令将 fib.o 链接为可执行文件 fib:

```
1 riscv64-linux-gnu-g++ fib.o -o fib -static
```

使用 `file` 命令查看生成的可执行文件，如图 18 所示，可以看到它是一个 RISC-V 架构的 ELF 可执行文件。

```
sun@sun-virtual-machine:~/Desktop$ file fib
fib: ELF 64-bit LSB pie executable, UCB RISC-V, RVC, double-float ABI, version 1
(SYSV), dynamically linked, interpreter /lib/ld-linux-riscv64-lp64d.so.1, Build
ID[sha1]=d02a924d4975cc1c778e3969482d640b32233e3b, for GNU/Linux 4.15.0, not str
tipped
```

图 18: `file` 命令查看可执行文件

我们使用 `objdump -d` 对 `fib` 进行反汇编，得到 `main` 函数的反汇编结果，如图 19 所示。可以看到，与汇编器阶段生成的目标文件不同，此时 `main` 函数的地址不再是 0，而是链接后分配的实际虚拟地址；`call` 指令的目标地址也已经被链接器填上。

```
0000000000000a08 <main>:
a08: 7179          addi    sp,sp,-48
a0a: f406          sd      ra,40(sp)
a0c: f022          sd      s0,32(sp)
a0e: 1800          addi    s0,sp,48
a10: 00001797      auipc   a5,0x1
a14: 6487b783      ld      a5,1608(a5) # 2058 <__stack_chk_guard@GLIBCXX_2.27>
a18: 6398          ld      a4,0(a5)
a1a: fee83423      sd      a4,-24(s0)
a1e: 4701          li      a4,0
a20: fc042c23      sw      zero,-40(s0)
a24: 4785          li      a5,1
a26: fcf42e23      sw      a5,-36(s0)
a2a: 4785          li      a5,1
a2c: fef42023      sw      a5,-32(s0)
a30: fd440793      addi    a5,s0,-44
a34: 85be          mv      a1,a5
a36: 00001517      auipc   a0,0x1
a3a: 65253503      ld      a0,1618(a0) # 2088 <ZSt3cin@GLIBCXX_3.4>
a3e: ea3ff0ef      jal     ra,800 <_ZNSt11__str_1@plt>
a42: fd842783      lw      a5,-40(s0)
a46: 85be          mv      a1,a5
a48: 00001517      auipc   a0,0x1
a4c: 63053503      ld      a0,1584(a0) # 2078 <ZSt4cout@GLIBCXX_3.4>
a50: ea1ff0ef      jal     ra,900 <_ZNSo15_E@plt>
a54: 87aa          mv      a5,a0
a56: 00001597      auipc   a1,0x1
a5a: 6125b583      ld      a1,1554(a1) # 2068 <ZSt4endlc11char_traits1cEERSt13basic_ostreamIT_0_E6_@GLIBCXX_3.4>
a5e: 853e          mv      a0,a5
a60: ea1ff0ef      jal     ra,900 <_ZNSo15_E@plt>
a64: fdc42783      lw      a5,-36(s0)
a68: 85be          mv      a1,a5
a6a: 00001517      auipc   a0,0x1
a6e: 60e53503      ld      a0,1550(a0) # 2078 <ZSt4cout@GLIBCXX_3.4>
a72: ebfff0ef      jal     ra,930 <_ZNSo15_E@plt>
a76: 87aa          mv      a5,a0
a78: 00001597      auipc   a1,0x1
a7c: 5f05b583      ld      a1,1520(a1) # 2068 <ZSt4endlc11char_traits1cEERSt13basic_ostreamIT_0_E6_@GLIBCXX_3.4>
a80: 853e          mv      a0,a5
a82: ea1ff0ef      jal     ra,900 <_ZNSo15_E@plt>
a86: a0b9          j       add <main+0xccc>
a88: fdc42783      lw      a5,-36(s0)
a8c: fef42223      sw      a5,-28(s0)
a90: fdc42783      lw      a5,-36(s0)
a94: 873e          mv      a4,a5
a96: fd842783      lw      a5,-40(s0)
a9a: 9fb9          addw    a5,a5,a4
a9c: fcf42e23      sw      a5,-36(s0)
aa0: fdc42783      lw      a5,-36(s0)
aa4: 85be          mv      a1,a5
aa6: 00001517      auipc   a0,0x1
aaa: 5d253503      ld      a0,1490(a0) # 2078 <ZSt4cout@GLIBCXX_3.4>
```

图 19: 链接后 `main` 函数反汇编

使用 `nm` 查看符号表，如图 20 所示，原来在目标文件中标记为 `U` 的未定义符号，在链接后已经被解析，或者标记为从动态库导入。

```

sun@sun-virtual-machine:~/Desktop$ riscv64-linux-gnu-nm fib
00000000000002b8 r __abi_tag
000000000000020b0 B __BSS_END__
000000000000020a0 B __bss_start
000000000000020a0 b completed.0
                                U __cxa_atexit@GLIBC_2.27
                                W __cxa_finalize@GLIBC_2.27
00000000000002000 D __DATA_BEGIN__
00000000000002000 D __data_start
00000000000002000 W data_start
000000000000097e t deregister_tm_clones
00000000000009cc t __do_global_ctors_aux
00000000000001de8 d __do_global_ctors_aux_fini_array_entry
00000000000002000 D __dso_handle
00000000000001df0 a _DYNAMIC
000000000000020a0 D _edata
000000000000020b0 B _end
0000000000000a06 t frame_dummy
00000000000001dd8 d __frame_dummy_init_array_entry
0000000000000c64 r __FRAME_END__
00000000000002050 a _GLOBAL_OFFSET_TABLE_
00000000000002800 A __global_pointer$

```

图 20: 链接后符号表

使用 `objdump -h` 查看段表，如图 21 所示。

```

sun@sun-virtual-machine:~/Desktop$ riscv64-linux-gnu-objdump -h fib

fib:      文件格式 elf64-littleriscv

节:
Idx Name           Size      VMA               LMA               File off  Algn
  0 .interp         00000021  0000000000000270  0000000000000270  00000270  2**0
                CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .note.gnu.build-id 00000024  0000000000000294  0000000000000294  00000294  2
**2
                CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .note.ABI-tag    00000020  00000000000002b8  00000000000002b8  000002b8  2**2
                CONTENTS, ALLOC, LOAD, READONLY, DATA
  3 .gnu.hash        00000024  00000000000002d8  00000000000002d8  000002d8  2**3
                CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .dynsym          000001b0  0000000000000300  0000000000000300  00000300  2**3
                CONTENTS, ALLOC, LOAD, READONLY, DATA
  5 .dynstr          00000183  00000000000004b0  00000000000004b0  000004b0  2**0
                CONTENTS, ALLOC, LOAD, READONLY, DATA
  6 .gnu.version     00000024  0000000000000634  0000000000000634  00000634  2**1
                CONTENTS, ALLOC, LOAD, READONLY, DATA
  7 .gnu.version_r   00000070  0000000000000658  0000000000000658  00000658  2**3
                CONTENTS, ALLOC, LOAD, READONLY, DATA
  8 .rela.dyn        00000150  00000000000006c8  00000000000006c8  000006c8  2**3

```

图 21: 链接后段表

进一步查看 `.plt` 节的反汇编，如图 22 所示。`.plt` (Procedure Linkage Table) 是用于动态链接的跳转指令表，包含了从程序代码到动态库中函数实现的间接跳转。链接器在链接时补上了之前的空白地址，实现了将目标文件和库文件链接到一起的功能。

```
sun@sun-virtual-machine:~/Desktop$ riscv64-linux-gnu-objdump -d -j .plt fib
fib:      文件格式 elf64-littleriscv

Disassembly of section .plt:

00000000000008c0 <.plt>:
8c0:  00001397          auipc  t2,0x1
8c4:  41c30333          sub   t1,t1,t3
8c8:  7483be03          ld    t3,1864(t2) # 2008 <__TMC_END__>
8cc:  fd430313          addi  t1,t1,-44
8d0:  74838293          addl  t0,t2,1864
8d4:  00135313          srli  t1,t1,0x1
8d8:  0082b283          ld    t0,8(t0)
8dc:  000e0067          jr    t3

00000000000008e0 <_ZNSirsERi@plt>:
8e0:  00001e17          auipc  t3,0x1
8e4:  738e3e03          ld    t3,1848(t3) # 2018 <_ZNSirsERi@GLIBCXX_3.4>
```

图 22: `.plt` 节反汇编

综上，链接器完成了符号解析和地址重定位，将多个目标文件和库文件合并成一个可执行文件，为程序的最终运行做好了准备。

(二) 熟悉 LLVM IR 中间语言

LLVM IR 是 LLVM 编译器框架的中间表示形式，具有类型化、可扩展、强表现力等特点。它有三种表示形式：内存中的对象、二进制 `bitcode` (`.bc`) 和可读的文本形式 (`.ll`)。本实验通过编写一个简单的 LLVM IR 程序，并链接 SysY 运行时库，来熟悉 LLVM IR 的基本语法与程序结构。

1. 实验程序

我们编写了一个计算阶乘的 LLVM IR 程序 `main.ll`，它调用 SysY 运行时库提供的 `getint`、`putint`、`putch` 函数完成输入输出。程序源码如图 23 所示。

```

declare i32 @getint()
declare void @putint(i32)
declare void @putch(i32)

define i32 @main() {
entry:
    %n = alloca i32
    %l = alloca i32
    %f = alloca i32
    %tmp = call i32 @getint()
    store i32 %tmp, i32* %n
    store i32 2, i32* %l
    store i32 1, i32* %f
    br label %loop

loop:
    %i_val = load i32, i32* %l
    %n_val = load i32, i32* %n
    %cond = icmp sle i32 %i_val, %n_val
    br i1 %cond, label %body, label %end

body:
    %l_cur = load i32, i32* %l
    %f_cur = load i32, i32* %f
    %f_new = mul nsw i32 %f_cur, %l_cur
    store i32 %f_new, i32* %f
    %l_new = add nsw i32 %l_cur, 1
    store i32 %l_new, i32* %l
    br label %loop

end:
    %result = load i32, i32* %f
    call void @putint(i32 %result)
    call void @putch(i32 10)
    ret i32 0
}

```

图 23: LLVM IR 阶乘程序 main.ll

2. 编译与链接

使用 clang 将 LLVM IR 文件编译为目标文件，再用 RISC-V 工具链链接 SysY 运行时库，生成可执行文件。命令如下：

```

1 clang --target=riscv64-unknown-linux-gnu -c main.ll -o main.o
2 riscv64-unknown-elf-gcc main.o ~/lib/libsysy_riscv.a -o main -static

```

生成的可执行文件 main 为 RISC-V 架构，如图 24 所示。

```

sun@sun-virtual-machine:~/Desktop$ clang --target=riscv64-unknown-linux-gnu -c main.ll -o main.o
warning: overriding the module target triple with riscv64-unknown-linux-gnu [-Woverride-module]
1 warning generated.
sun@sun-virtual-machine:~/Desktop$ riscv64-unknown-elf-gcc main.o ~/lib/libsysy_riscv.a -o main -static
sun@sun-virtual-machine:~/Desktop$ file main
main: ELF 64-bit LSB executable, UCB RISC-V, RVC, double-float ABI, version 1 (SYSV), statically linked, with debug_info, not stripped

```

图 24: 编译链接命令及 file 命令查看可执行文件

3. 运行验证

使用 QEMU 运行程序，输入 5，程序输出 120，结果正确，如图 25 所示。

```

sun@sun-virtual-machine:~/Desktop$ echo 5 | qemu-riscv64 ./main
120
TOTAL: 0H-0M-0S-0us

```

图 25: LLVM IR 程序运行结果

通过本实验，我们了解了 LLVM IR 的基本结构，包括函数定义、基本块、虚拟寄存器、类型系统、分支与指令等，并掌握了将 LLVM IR 与 SysY 运行时库链接并执行的方法。

（三）熟悉汇编语言

以斐波那契数列程序为例，使用 RISC-V 汇编语言实现与 C 程序相同的功能。源程序 fibonacci.c 如下：

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a, b, i, t, n;
6
7      a = 0;
8      b = 1;
9      i = 1;
10
11     scanf("%d", &n);
12
13     printf("%d\n", a);
14     printf("%d\n", b);
15
16     while (i < n)
17     {
18         t = b;
19         b = a + b;
20         printf("%d\n", b);
21         a = t;
22         i = i + 1;
23     }
24
25     return 0;
26 }
```

RISC-V 汇编实现

主要寄存器分配如下：

寄存器	保存内容
s0	变量 a
s1	变量 b
s2	变量 i
s3	临时变量 t
s4	输入变量 n
a0、a1	函数参数
ra	返回地址

完整的 RISC-V 汇编程序 `fibonacci.s` 如下：

```
1      .option nopic
2      .attribute arch, "rv64gc"
3      .attribute unaligned_access, 0
4      .attribute stack_align, 16
5
6      .section .rodata
7      .align 3
8      .LC0:
9          .string "%d"
10         .align 3
11     .LC1:
12         .string "%d\n"
13
14     .text
15     .align 1
16     .globl main
17     .type main, @function
18
19 main:
20     addi sp, sp, -64
21     sd ra, 56(sp)
22     sd s0, 48(sp)
23     sd s1, 40(sp)
24     sd s2, 32(sp)
25     sd s3, 24(sp)
26     sd s4, 16(sp)
27
28     # a = 0, b = 1, i = 1
29     li s0, 0
30     li s1, 1
31     li s2, 1
32
33     # scanf("%d", &n)
34     lui a0, %hi(.LC0)
35     addi a0, a0, %lo(.LC0)
36     addi a1, sp, 8
37     call scanf
38     lw s4, 8(sp)
39
40     # 输出初始值 a
41     lui a0, %hi(.LC1)
42     addi a0, a0, %lo(.LC1)
43     mv a1, s0
```

```

44     call printf
45
46     # 输出初始值 b
47     lui a0, %hi(.LC1)
48     addi a0, a0, %lo(.LC1)
49     mv a1, s1
50     call printf
51
52 .Lloop:
53     # if (i >= n) 结束循环
54     bge s2, s4, .Lend
55
56     # t = b; b = a + b
57     mv s3, s1
58     add s1, s0, s1
59
60     # printf("%d\n", b)
61     lui a0, %hi(.LC1)
62     addi a0, a0, %lo(.LC1)
63     mv a1, s1
64     call printf
65
66     # a = t; i = i + 1
67     mv s0, s3
68     addi s2, s2, 1
69     j .Lloop
70
71 .Lend:
72     # return 0
73     li a0, 0
74
75     ld ra, 56(sp)
76     ld s0, 48(sp)
77     ld s1, 40(sp)
78     ld s2, 32(sp)
79     ld s3, 24(sp)
80     ld s4, 16(sp)
81     addi sp, sp, 64
82     ret
83
84     .size main, .-main
85     .section .note.GNU-stack,"",@progbits

```

程序进入 main 后, 首先将 ra 和 s0 到 s4 保存到栈中, 再完成变量初始化。调用 scanf 时, a0 保存格式字符串 "%d" 的地址, a1 保存变量 n 在栈中的地址; 输入完成后, lw s4, 8(sp) 将

n 的值读入寄存器 s4。两次 printf 的调用方式相同，a0 传入格式字符串，a1 传入待输出的整数值。

循环开始处的 bge s2, s4, .Lend 表示当 $i \geq n$ 时退出循环，因此等价于 C 语言中的 while ($i < n$)。循环体中，mv s3, s1 保存旧的 b，add s1, s0, s1 完成 $b = a + b$ ；输出新的 b 后，mv s0, s3 和 addi s2, s2, 1 分别完成 $a = t$ 和 $i = i + 1$ 。最后使用 j .Lloop 跳回循环条件处。

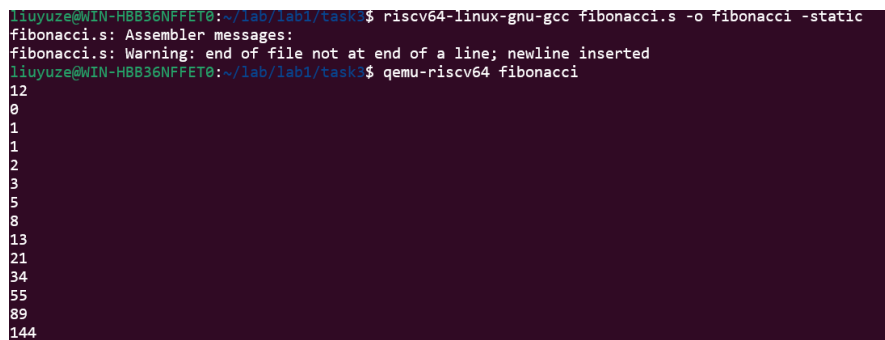
程序结束时，将 a0 置为 0 作为返回值，依次恢复保存过的寄存器，使用 addi sp, sp, 64 释放栈空间，最后通过 ret 返回调用者。

运行验证

使用 RISC-V 交叉编译器和 QEMU 运行程序：

```
fibonacci.s -o fibonacci -static
qemu-riscv64 fibonacci
```

输入 12 后，终端依次输出从 0 开始的斐波那契数列，结果如图 26 所示。



```
liuyuze@WIN-H8B36NFFET0: ~/lab/lab1/task$ riscv64-linux-gnu-gcc fibonacci.s -o fibonacci -static
fibonacci.s: Assembler messages:
fibonacci.s: Warning: end of file not at end of a line; newline inserted
liuyuze@WIN-H8B36NFFET0: ~/lab/lab1/task$ qemu-riscv64 fibonacci
12
0
1
1
2
3
5
8
13
21
34
55
89
144
```

图 26: 斐波那契汇编程序的运行结果

汇编程序中的变量初始化、输入、循环计算和输出均正确。

反汇编验证

为了观察汇编器最终生成的目标指令，执行：

```
riscv64-linux-gnu-objdump -d fibonacci \
| sed -n '/<main>:./,/^$/p'
```

反汇编结果如图 27 所示。

```

liuyuze@WIN-HBB36NFFET0:~/lab/lab1/task3$ riscv64-linux-gnu-objdump -d fibonacci | sed -n '/<main>:/,/^$/p'
0000000001062a <main>:
1062a: 7139          addi    sp,sp,-64
1062c: fc06          sd      ra,56(sp)
1062e: f822          sd      s0,48(sp)
10630: f426          sd      s1,40(sp)
10632: f04a          sd      s2,32(sp)
10634: ec4e          sd      s3,24(sp)
10636: e852          sd      s4,16(sp)
10638: 4401          li      s0,0
1063a: 4485          li      s1,1
1063c: 4905          li      s2,1
1063e: 0005e537      lui     a0,0x5e
10642: 09850513      addi    a0,a0,152 # 5e098 <_IO_stdin_used+0x8>
10646: 002c          addi    a1,sp,8
10648: 2ad040ef      jal     ra,150f4 <__scanf>
1064c: 4a22          lw      s4,8(sp)
1064e: 0005e537      lui     a0,0x5e
10652: 0a050513      addi    a0,a0,160 # 5e0a0 <_IO_stdin_used+0x10>
10656: 85a2          mv      a1,s0
10658: 205040ef      jal     ra,1505c <_IO_printf>
1065c: 0005e537      lui     a0,0x5e
10660: 0a050513      addi    a0,a0,160 # 5e0a0 <_IO_stdin_used+0x10>
10664: 85a6          mv      a1,s1
10666: 1f7040ef      jal     ra,1505c <_IO_printf>
1066a: 01495e63      bge     s2,s4,10686 <main+0x5c>
1066e: 89a6          mv      s3,s1
10670: 94a2          add     s1,s1,s0
10672: 0005e537      lui     a0,0x5e
10676: 0a050513      addi    a0,a0,160 # 5e0a0 <_IO_stdin_used+0x10>
1067a: 85a6          mv      a1,s1
1067c: 1e1040ef      jal     ra,1505c <_IO_printf>
10680: 844e          mv      s0,s3
10682: 0905          addi    s2,s2,1
10684: b7dd          j       1066a <main+0x40>
10686: 4501          li      a0,0
10688: 70e2          ld      ra,56(sp)
1068a: 7442          ld      s0,48(sp)
1068c: 74a2          ld      s1,40(sp)
1068e: 7902          ld      s2,32(sp)
10690: 69e2          ld      s3,24(sp)
10692: 6a42          ld      s4,16(sp)
10694: 6121          addi    sp,sp,64
10696: 8082          ret

```

图 27: 斐波那契程序的反汇编结果

反汇编结果与编写的汇编程序一致: `li` 指令完成变量初值设置, `jal` 调用 `scanf` 和 `printf`, `bge` 判断循环是否结束, `add`、`addi`、`mv` 完成斐波那契数列的计算和变量更新。由于程序采用静态链接, 反汇编中的 `scanf` 和 `printf` 分别显示为 `__scanf` 和 `_IO_printf`, 但其调用功能不变。返回前, `ld` 指令恢复保存的寄存器, `addi sp, sp, 64` 释放栈帧, 最后通过 `ret` 返回。

三、 总结

本次实验从完整编译流程出发，借助 GCC/Clang、LLVM 和 RISC-V 工具链分别完成了预处理、编译、汇编和链接的观察与验证。在编译器内部，我们结合词法分析、语法分析、语义分析、LLVM IR 生成以及不同优化级别的结果，理解了源代码到中间表示的转换；在目标代码阶段，则通过汇编、反汇编、符号表和重定位信息，梳理了目标文件与可执行文件之间的联系。最终，LLVM IR 阶乘程序和 RISC-V 斐波那契程序均运行正确，使我们对编译系统各阶段的分工与协作有了更直观、完整的认识。

仓库链接: <https://github.com/liuyuze-121/nku-compiler-labs.git>

NKU